

Luke Jeffries
AI-CS-477
Prof. Mongon

Schedule Organizer

The motivation for this project was my desire to create a tool that could help me better organize my workload during finals week. Finals are consistently challenging to keep up with, especially when trying to balance multiple assignments with different workloads and deadlines. To address this, I decided to develop an AI-based system that could assist with planning by estimating how long different tasks would take to complete. Over time, the system is intended to learn from my habits and past completion times, allowing it to generate increasingly accurate and realistic schedules for future weeks.

Dataset

The project uses an artificially generated task dataset, created with a custom script (training/generate_data.py). Each data point contains a short natural-language task description (e.g., “Write a 3–4 page analysis paper comparing two historical interpretations”) and a corresponding numeric label representing the estimated completion time in hours.

```
def generate_task(template, hours, noise=0.5):  
    noisy_hours = max(0.5, hours + np.random.uniform(- noise, noise))  
    return {  
        "description": template,  
        "hours": round(noisy_hours, 2)  
    }
```

Synthetic data was chosen deliberately because it is controlled and reproducible, free from sensitive or personal student information, and flexible enough to represent a wide range of assignment types. This allowed experimentation without the ethical concerns associated with scraping real student work or monitoring individual behavior.

Bias and Ethics

While synthetic data eliminates privacy concerns, it introduces template bias, where the model may associate certain keywords (such as “paper” or “final”) with fixed time estimates due to repeated phrasing patterns. To mitigate this, multiple templates were used for similar task types, small random noise was added to the time labels, and the system was framed as a planning aid rather than a definitive measure of effort. Any real-world deployment would require explicit user consent and personalization features.

Data Preparation and Feature Engineering

Task descriptions were converted into numerical features using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization, which captures how important words and phrases are within a task relative to the entire dataset.

```
vectorizer = TfidfVectorizer(  
    max_features=3000,  
    ngram_range=(1, 2),  
    stop_words="english"  
)
```

```
X_text = vectorizer.fit_transform(df["description"])
```

To supplement textual information, structural features were added to approximate task complexity.

```
df["word_count"] = df["description"].apply(lambda x: len(x.split()))  
df["char_count"] = df["description"].apply(len)
```

These features help capture task scope, which often correlates with workload.

Modeling and Selection

Models Evaluated

Three regression models were tested: Linear Regression as a simple baseline, Random Forest Regressor for modeling nonlinear relationships, and a

Multi-Layer Perceptron (MLP) for flexible representation learning. Each model was evaluated for accuracy, stability, and interpretability.

Final Model Selection

The Random Forest Regressor was selected as the final model due to its consistent performance and robustness.

```
model = RandomForestRegressor(  
    n_estimators=200,  
    random_state=42  
)
```

```
model.fit(X_train, y_train)
```

This model captured complex patterns more effectively than Linear Regression while requiring less tuning and providing greater stability than the MLP.

Evaluation Methodology

Metrics

Model performance was evaluated using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE), which align well with the goal of estimating task duration.

```
preds = model.predict(X_test)
```

```
mae = mean_absolute_error(y_test, preds)
```

```
rmse = mean_squared_error(y_test, preds, squared=False)
```

Explainability and Interpretation

Motivation

For a system designed to support student planning, transparency is essential. Users must be able to understand why a task is estimated to take a certain amount of time.

SHAP Analysis

SHAP was used to interpret the Random Forest model.

```
explainer = shap.TreeExplainer(model)  
shap_values = explainer.shap_values(X_sample)
```

The analysis showed that terms such as “final,” “research,” “pages,” and “compare” increased predicted completion time, while words like “quiz” or “review” reduced estimates. Structural features such as word count provided a stable baseline influence.

Limitations and Future Directions

Limitations

The system does not yet account for individual differences in working habits, and phrasing may not fully represent real-world assignments.

Future Work

Future improvements could include incorporating user feedback, personalizing estimates based on past completion times, exporting schedules directly to calendar formats (such as iCal or Google Calendar), and evaluating the system on anonymized real-world data with appropriate consent.

Conclusion

This project demonstrates that machine learning can accurately and interpretably estimate assignment workload from concise text descriptions. By combining TF-IDF features, Random Forest modeling, and SHAP-based explainability, the system provides a strong foundation for intelligent academic planning tools with potential for further personalization and automation.